

DAYS 12 – 14

Enterprise Capstone Project

Multi-Tenant Inventory & Workflow Management API — build it from scratch, ship it like a professional

No study sessions for Days 12–14. If you are stuck for more than 20 minutes, check your notes from Days 0–11 first. Searching your own knowledge is how real engineering works.

This project must be built from a blank repository — not from the exercises. You must remember every setup step: pnpm init, NestJS scaffold, Fastify adapter, PrismaService, environment variables, main.ts bootstrap. This is exactly what you do when building a new service on a real project.

Project Brief

Build a production-ready multi-tenant inventory and workflow management API for an enterprise SaaS platform. This API would be one of several services in a real product. It must meet every standard introduced in Days 0–11.

Functional Requirements

Requirement	Description	Standards
Multi-Tenancy	All data is company-scoped. Every query filters by companyId from CompanyContext. Cross-tenant access must be architecturally impossible.	Repository enforces companyId on every query. Test: Company A cannot see Company B's data.
User Management	Users belong to companies. Roles: ADMIN, MANAGER, VIEWER. ADMIN manages users and roles. MANAGER manages inventory. VIEWER reads only.	RBAC guard on all write operations. Role stored in JWT. RolesGuard validates on every request.
Inventory Items	CRUD: title, description, SKU, quantity, status (IN_STOCK / LOW_STOCK / OUT_OF_STOCK), category. All fields validated. Soft delete.	SKU unique per company. Status auto-updated by quantity. deletedAt excludes from all queries.
Inventory Adjustments	Adjustment endpoint: add or remove quantity with a reason. Each adjustment writes an AuditLog record in the same transaction.	Atomic transaction: quantity update + AuditLog. Concurrent adjustment test: no negative quantity.
Advanced Queries	Filter by status, category, quantity range (min/max). Full-text search on title. Cursor-based pagination. Multi-field sort.	All filters composable. Cursor stable under concurrent writes. companyId always included.
Low-Stock Alerts	When quantity drops below configurable threshold, a BullMQ job is enqueued. Worker logs a structured alert with item details.	Job enqueued in same transaction as adjustment. Worker retries on failure. Dead-letter queue for poison messages.

Requirement	Description	Standards
Redis Caching	Inventory summary endpoint (count by status) is cached per company. TTL: 60s. Cache invalidated on any item write.	Cache key includes companyId. Invalidation on create/update/delete. Test: stale data is never served after write.

Technical Requirements

- Authentication: JWT with userId, companyId, role in payload. Register and login endpoints. Passwords hashed with bcrypt.
- Authorization: RolesGuard + @Roles() decorator. VIEWER cannot mutate. MANAGER cannot modify users. ADMIN has full access.
- API versioning: all routes under /v1/. API prefix: /api.
- Response envelope: { data, meta?, errors? } on all responses. Never return raw Prisma objects.
- Error handling: ApplicationException hierarchy. All errors return { statusCode, errorCode, message, requestId, timestamp }.
- Structured logging: Pino with AsyncLocalStorage. Every log line includes requestId, userId, companyId, action.
- Health checks: GET /v1/health and GET /v1/health/ready with real DB and Redis connectivity checks.
- Swagger documentation: all endpoints documented, all DTOs decorated with @ApiProperty. Accessible at /api.
- Soft delete: all items use deletedAt. Deleted items excluded from all queries by Prisma middleware.
- Audit log: every create/update/delete writes an immutable AuditLog record in the same transaction.
- Docker: multi-stage Dockerfile (non-root user). docker-compose.yml with PostgreSQL and Redis. HEALTHCHECK configured.
- CI/CD: GitHub Actions workflow with lint, unit test, integration test, build, e2e test jobs. Branch protection on main.
- Testing: unit tests for all services (mocked repositories). Integration tests for all repositories (real test DB). E2E tests for all critical paths. 80% coverage threshold enforced.
- README: project description, prerequisites, setup instructions, environment variables (every one documented), how to run locally, how to run tests, endpoint list.

Day-by-Day Plan

Day	Focus	Deliverables	Definition of Done
Day 12	Architecture & Core Build	Blank repo setup. Prisma schema. All modules with repository pattern. Auth with RBAC. Full request pipeline. Swagger.	All CRUD endpoints working. JWT auth + RBAC enforced. Swagger live at /api. Postman collection built.
Day 13	Advanced Features & Infrastructure	Cursor pagination. Filters + search. Inventory adjustment with AuditLog transaction. Redis caching. BullMQ low-stock job. Docker.	Full Postman flow works end-to-end. Docker compose up starts successfully. Health check passes.
Day 14	Tests, CI/CD, README & PR	Unit + integration + e2e tests. GitHub Actions CI pipeline. README. Self-review. PR opened.	All tests pass. Coverage >= 80%. CI pipeline green. PR description written. Self-review done.

Days 12–14 — Enterprise Capstone

- ☐ Blank repo set up with all standards from Days 0–11 applied from the start
- ☐ Full Prisma schema: Company, User, Item, AuditLog — all with correct indexes and relations
- ☐ All modules with correct layered structure: Controller → Service → IRepository → PrismaRepository
- ☐ JWT auth with RBAC: register, login, role enforcement on all write operations
- ☐ Full request pipeline: RequestContextMiddleware, LoggingInterceptor, CompanyContextGuard, GlobalExceptionHandler
- ☐ All functional requirements met: CRUD, adjustment, filters, pagination, caching, background jobs
- ☐ Docker: multi-stage Dockerfile, docker-compose.yml, HEALTHCHECK, .env.example
- ☐ CI/CD: GitHub Actions pipeline all green — lint, test, build, e2e, coverage gate
- ☐ Tests: unit + integration + e2e passing. Coverage $\geq$ 80% on service layer.
- ☐ README: setup, env vars, how to run, how to test, endpoint list — can be followed by a stranger
- ☐ PR description written: what it does, architecture decisions, how to test it locally, known limitations
- ☐ Self-review completed — diff read as if you did not write it
- ☐ PR opened, CI green, ready for senior engineer code review